

Step 1 — A Normal Graph With Only Short-Term (Checkpoint) Memory — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume a `.env` with `OPENAI_API_KEY` set, and are run from inside `project_10_memorykeeper_persistent_memory/`.

Basic Version

Approach 1 — the direct way, one file, no persistence yet

```
# main.py (Basic -- no checkpoint at all, just to see the gap)
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini")
messages = []

print("MemoryKeeper (no memory yet). Type 'exit' to quit.")
while True:
    text = input("you: ")
    if text.strip().lower() == "exit":
        break
    messages.append({"role": "user", "content": text})
    reply = model.invoke(messages)
    messages.append(reply)
    print("bot:", reply.content)
```

Expected output: within one run, this remembers fine -- `messages` is just a growing Python list. Close the program and run it again: `messages` starts empty, every time, because nothing was ever saved anywhere. This is Doc04's chatbot exactly -- no checkpoint, no graph, memory that only ever exists inside one running process.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a real LangGraph graph, with a file-backed checkpointer

```
# state.py
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages
```

```
class ChatState(TypedDict):
    messages: Annotated[list, add_messages]
```

```
# graph.py
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from state import ChatState

model = ChatOpenAI(model="gpt-4o-mini")

def chat_node(state: ChatState) -> dict:
    reply = model.invoke(state["messages"])
    return {"messages": [reply]}

def build_graph(checkpointer):
    builder = StateGraph(ChatState)
    builder.add_node("chat_node", chat_node)
    builder.add_edge(START, "chat_node")
    builder.add_edge("chat_node", END)
    return builder.compile(checkpointer=checkpointer)
```

```
# main.py
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import build_graph

THREAD_ID = "sam-thread-1"
```

```
def main():
    conn = sqlite3.connect("memory_keeper.db", check_same_thread=False)
    checkpointer = SqliteSaver(conn)
    graph = build_graph(checkpointer)

    print(f"MemoryKeeper -- thread_id={THREAD_ID}. Type 'exit' to quit.")
    while True:
        text = input("you: ")
        if text.strip().lower() == "exit":
            break
        config = {"configurable": {"thread_id": THREAD_ID}}
        result = graph.invoke({"messages": [{"role": "user", "content": text}]},
                               print("bot:", result["messages"][-1].content))

if __name__ == "__main__":
    main()
```

Expected output: run `python main.py`, say `my name is Sam`, type `exit`. Run `python main.py` again -- same `THREAD_ID` still in the file -- ask `what's my name?`. It answers correctly, because `memory_keeper.db` really has that thread's messages saved on disk.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a real test script proving both halves of the baseline

```
# test_memory_keeper.py
import subprocess
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import build_graph

def ask(thread_id: str, text: str) -> str:
    conn = sqlite3.connect("memory_keeper.db", check_same_thread=False)
    checkpointer = SqliteSaver(conn)
    graph = build_graph(checkpointer)
    config = {"configurable": {"thread_id": thread_id}}
    result = graph.invoke({"messages": [{"role": "user", "content": text}]}, config)
    return result["messages"][-1].content

def test_same_thread_remembers():
    ask("test-thread-a", "My name is Sam. Remember it.")
    answer = ask("test-thread-a", "What is my name?")
    assert "sam" in answer.lower()

def test_new_thread_forgets():
    ask("test-thread-b", "My name is Priya. Remember it.")
    # a completely different thread_id -- brand new, empty state on purpose
    answer = ask("test-thread-c", "What is my name?")
    assert "priya" not in answer.lower()

if __name__ == "__main__":
    test_same_thread_remembers()
    print("PASS: same thread_id remembers across calls")
    test_new_thread_forgets()
    print("PASS: a different thread_id has no memory of the other one")
```

Expected output:

PASS: same `thread_id` remembers across calls

PASS: a different `thread_id` has no memory of the other one

Each `ask()` call opens its own fresh `sqlite3.connect(...)` and rebuilds the graph -- this is deliberately closer to "a separate process" than reusing one connection across the whole test file, since a shared in-Python connection object could hide a bug that a real process restart would expose. For the strongest version of this proof, run `test_same_thread_remembers`'s two `ask()` calls as two separate `python -c "..."` invocations instead of two calls in one test function -- that removes any chance the result is coming from Python objects still sitting in memory rather than the actual `.db` file.

Difference from Intermediate, and what this approach adds: Intermediate proves the happy path by hand, once, by actually re-running `main.py` yourself. Advanced turns that into a repeatable, automated test that checks *both* directions on purpose -- a repeated `thread_id` remembering, and a fresh one not -- so a future change to this project can't quietly break either guarantee without a test catching it.

Which one should you actually write? Build Intermediate's `graph.py / state.py / main.py` exactly as shown -- this is the real shape every later step adds to, unchanged. Then write Advanced's test file, even though it's more code than the step strictly requires: it's what actually proves the baseline this whole project measures itself against, and it's the test you'll reuse (with more assertions added) in every step after this one.